

Advanced Python Programming For Data Science

Chapter 2: Data Sources and APIs

Chapter Overview

This chapter focuses on how data is:

- Collected
- Accessed
- Processed
- Prepared for analysis

Duration: 7 hours

Marks Weightage: 9 Marks

Learning Outcomes

After completing this chapter, students will be able to:

- Read and write data from multiple file formats
- Access data from relational and non-relational databases
- Consume and process data from APIs
- Extract data from web pages
- Handle large datasets efficiently

Chapter Structure

1. Reading and writing data (CSV, JSON, Excel, text)
2. Database access (Relational & Non-Relational)
3. Accessing data from APIs (REST, SOAP)
4. Web scraping
5. Handling large datasets

2.1 Reading and writing structured/unstructured data (CSV, JSON, Excel, text)

You will learn to:

- Read and write:
 - CSV
 - JSON
 - Excel
 - Text files
- Understand:
 - Structured vs unstructured data
- Choose the right format for the task

Structured vs Unstructured Data

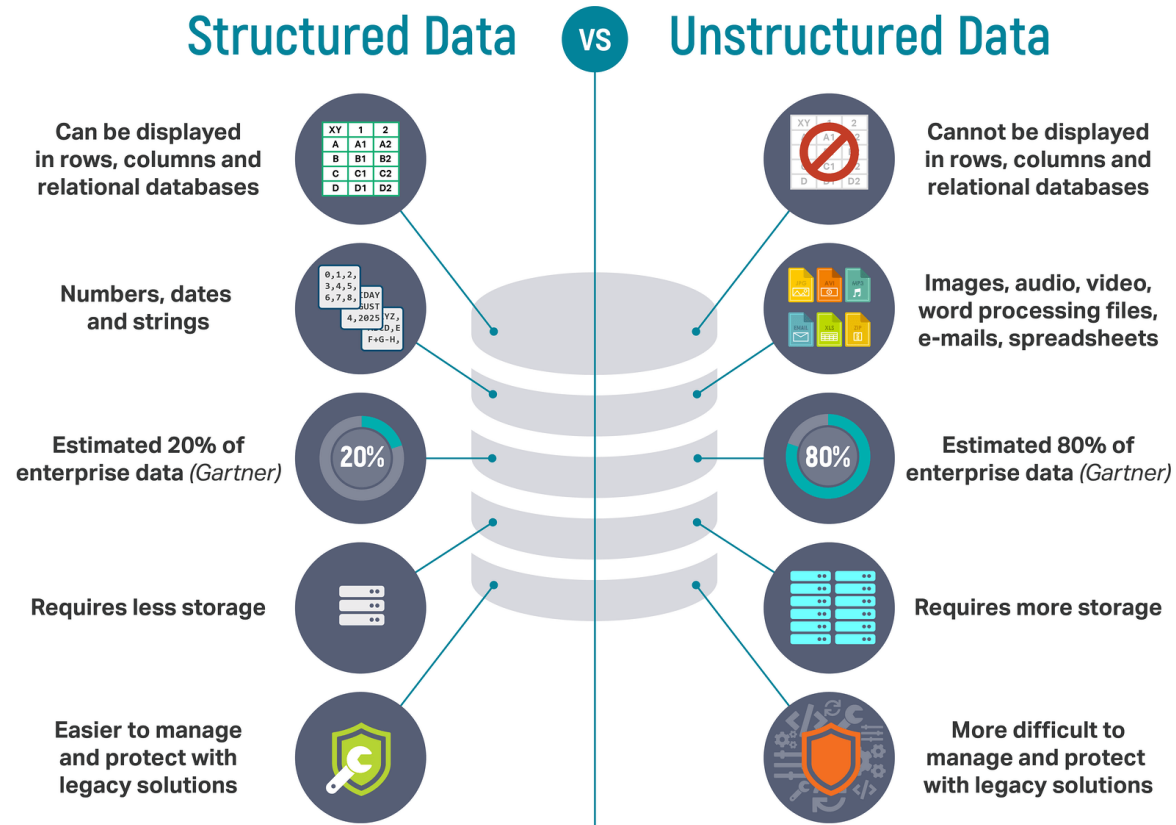
Structured Data

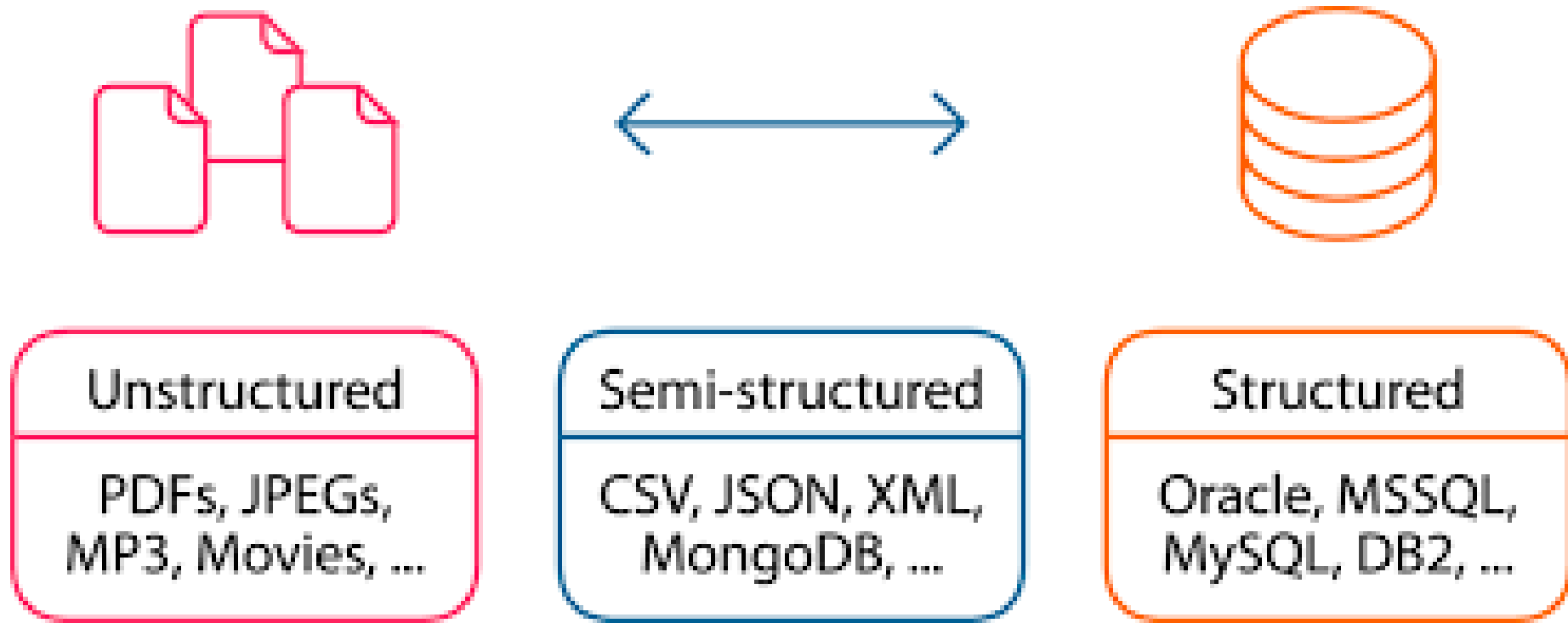
- Follows a fixed schema
- Organized in rows and columns
- Examples:
 - CSV
 - Excel
 - Database tables

Unstructured Data

- No fixed format
- Text-heavy or semi-structured
- Examples:
 - Text files
 - Multimedia (Photos, Audio/Video files)
 - Logs

Structured vs Unstructured Data





Structured Data: Data stored in a fixed schema with rows and columns.

Semi-Structured Data: Data with flexible structure using tags or key-value pairs.

Unstructured Data: Data without any predefined format or structure.



Difference Between Structured, Semi-Structured, and Unstructured Data

Aspect	Structured Data	Semi-Structured Data	Unstructured Data
Definition	Data organized in a fixed tabular structure	Data with partial structure using tags or keys	Data with no predefined structure
Schema	Strict, predefined schema	Flexible or evolving schema	No schema
Data Format	Rows and columns	Hierarchical / key–value	Free-form
Organization	Highly organized	Moderately organized	Unorganized
Storage System	Relational databases	NoSQL databases, files	File systems, document stores
Querying	Easy querying using SQL	Limited querying with parsing	No direct querying
Processing Difficulty	Easy to process	Moderately difficult	Difficult and complex
Examples	CSV, Excel, database tables	JSON, XML	Text, images, audio, video
Data Integrity	High integrity constraints	Partial constraints	No constraints
Typical Use Case	Transaction systems, reporting	APIs, logs, configurations	Social media, documents, multimedia

© 2006 The Authors

```
"orders": [
  {
    "orderno": "748745375",
    "date": "June 30, 2088 1:54:23 AM",
    "trackingno": "TN0039291",
    "custid": "11045",
    "customer": [
      {
        "custid": "11045",
        "fname": "Sue",
        "lname": "Hatfield",
        "address": "1409 Silver Street",
        "city": "Ashland",
        "state": "NE",
        "zip": "68003"
      }
    ]
  }
]
```

B13			=SUM(B3:B12)		
	A	B	C	D	E
1		Sales in Each Quarter			
2	Product Name	Jan'2018	April'2018	July'2018	October'2018
3	ABC Mutton	\$ 2,667.60	\$ 4,013.10	\$ 4,836.00	\$ 6,087.90
4	Crab Meat	\$ 1,768.41	\$ 1,978.00	\$ 4,412.32	\$ 1,656.00
5	Camembert Pierrot	\$ 3,182.40	\$ 4,683.50	\$ 9,579.50	\$ 3,060.00
6	Ipoh Coffee	\$ 1,398.40	\$ 4,496.50	\$ 1,196.00	\$ 3,979.00
7	Hot Pepper Sauce	\$ 1,347.36	\$ 2,750.69	\$ 1,375.62	\$ 3,899.51
8	Hot Spiced Okra	\$ 1,509.60	\$ 530.40	\$ 68.00	\$ 850.00
9	Mozzarella di Giovanni	\$ 1,390.00	\$ 4,488.20	\$ 3,027.60	\$ 2,697.00
10	Sir Rodney's Scones	\$ 1,462.00	\$ 644.00	\$ 1,733.00	\$ 1,434.00
11	Steeleye Stout	\$ 1,310.40	\$ 1,368.00	\$ 1,323.00	\$ 1,273.50
12	Veggie-spread	\$ 3,202.87	\$ 263.40	\$ 842.88	\$ 2,590.10
13	Grand Total	\$ 19,239.04			
14					

A	B	C	D	E	F	G
	Price	Description	Ingredients	Image link		
uct						
l True Mass	35\$	TRUE-MASS- Protein Matr		https://www.megaproteine.ro/img/pro		
l STANDARD 100% Whey	80\$	Whey Protein Protein Bleni		https://www.megaproteine.ro/img/pro		
ation Xtend BCAAs Amino acids	20\$	Scivation XTI Citric Acid ,		https://www.megaproteine.ro/img/pro		
icle Line L-Carnitine	10\$	L-Carnitine is Magnesium		https://www.megaproteine.ro/img/pro		
icleTech Cell-Tech Hyper-Build	25\$	CELL-TECH is Silicon Dioxid		https://www.megaproteine.ro/img/pro		
workout stimulant N.O.-Xplode XE Edge	10\$	N.O.-XPLODE Acids (Citric		https://www.megaproteine.ro/img/pro		
icle Line Multivitamins	12\$	Multivitamin Calcium Carb		https://www.megaproteine.ro/img/pro		
st Protein Bar Blueberry Muffin	2\$	The Quest Bl Protein Bleni		https://www.megaproteine.ro/img/pro		
nal Flex	15\$	Animal Flex Dicalcium Ph		https://www.megaproteine.ro/img/pro		
iversum Wshape Collagen	25\$	Collagen is a Collagen (frc		https://www.megaproteine.ro/img/pro		
ures Aid Hyaluronic Acid	18\$	Natures Aid Di-Calcium P		https://www.megaproteine.ro/img/pro		
ramino Sports Fuel	5\$	NUTRAMINC Natural min		https://www.megaproteine.ro/img/pro		
1 N Time Meal Replacement Shake	22\$	Diet Meal Re Protein Bleni		https://www.megaproteine.ro/img/pro		
1 N Time Skinny Juice	16\$	A refreshing, Maltodextrin		https://www.megaproteine.ro/img/pro		
ein Snack Mix Organic	4\$	High-protein soyabeans*		https://www.megaproteine.ro/img/pro		
rmo Cuts Fat Burner - Optimum Nutrition	18\$	Thermo Cuts N-Acetyl L-C		https://www.megaproteine.ro/img/pro		
thermo Speed Extreme Mega Caps	22\$	Thermo Speed 41% l-tyrosin		https://www.megaproteine.ro/img/pro		
l STANDARD 100% Casein	34\$	Fastest digest! Micellar Casi		https://www.megaproteine.ro/img/pro		
y Protein Complex		Olímp Pure V 95% protein-		https://www.megaproteine.ro/img/pro		
inum 100% L-Arginine	6\$	Platinum 100 Microcrystal		https://www.megaproteine.ro/img/pro		

```

[11/Dec/2018:11:01:28 -0600] "GET /blog/seo/robots-txt-mistakes-best-uses-gu
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/themes/portent_portent/assets/
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/themes/portent_portent/assets/
[11/Dec/2018:11:01:28 -0600] "GET /about/careers HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:28 -0600] "GET /blog/internet-marketing/the_internet_mark
[11/Dec/2018:11:01:28 -0600] "GET /blog/featured HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:28 -0600] "GET /services/ppc HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:28 -0600] "GET /services/amazon HTTP/1.1" 502 182 "-" "Sc
[11/Dec/2018:11:01:28 -0600] "GET /case-study/expand-audience-reach-content-
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/themes/portent_portent/assets/
[11/Dec/2018:11:01:28 -0600] "GET /blog/seo/structure-site-navigation-for-seo
[11/Dec/2018:11:01:28 -0600] "GET /blog/author/caleb-cosper HTTP/1.1" 502 18
[11/Dec/2018:11:01:28 -0600] "GET /about HTTP/1.1" 502 182 "-" "Screaming Fr
[11/Dec/2018:11:01:28 -0600] "GET /contact HTTP/1.1" 502 182 "-" "Screaming
[11/Dec/2018:11:01:28 -0600] "GET /case-study/ppc-campaign-restructure-fuels
[11/Dec/2018:11:01:28 -0600] "GET /about/philosophy HTTP/1.1" 502 182 "-" "S
[11/Dec/2018:11:01:28 -0600] "GET /services/seo HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:28 -0600] "GET /services/smb HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/themes/portent_portent/assets/
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/cache/autoptimize/css/autoptimi
[11/Dec/2018:11:01:28 -0600] "GET /resources HTTP/1.1" 502 182 "-" "Screamin
[11/Dec/2018:11:01:28 -0600] "GET /tools HTTP/1.1" 502 182 "-" "Screaming Fr
[11/Dec/2018:11:01:28 -0600] "GET /wp-content/cache/autoptimize/css/autoptimi
[11/Dec/2018:11:01:28 -0600] "GET /blog/seo HTTP/1.1" 502 182 "-" "Screaming
[11/Dec/2018:11:01:28 -0600] "GET /services/content HTTP/1.1" 502 182 "-" "S
[11/Dec/2018:11:01:29 -0600] "GET /services/analytics HTTP/1.1" 502 182 "-"
[11/Dec/2018:11:01:29 -0600] "GET /privacy HTTP/1.1" 502 182 "-" "Screaming
[11/Dec/2018:11:01:29 -0600] "GET /blog/internet-marketing HTTP/1.1" 502 18
[11/Dec/2018:11:01:29 -0600] "GET /wp-content/cache/autoptimize/js/autoptimi
[11/Dec/2018:11:01:29 -0600] "GET /blog HTTP/1.1" 502 182 "-" "Screaming Fro
[11/Dec/2018:11:01:29 -0600] "GET /ebook/seo/technical-seo-best-practices-the
[11/Dec/2018:11:01:29 -0600] "GET /case-studies HTTP/1.1" 502 182 "-" "Scree
[11/Dec/2018:11:01:29 -0600] "GET /case-study/linode-cloud-hosting-lead-gene
[11/Dec/2018:11:01:29 -0600] "GET /blog/author/ian HTTP/1.1" 502 182 "-" "Sc
[11/Dec/2018:11:01:29 -0600] "GET /blog/internet-marketing/stack-overview.Ht
[11/Dec/2018:11:01:29 -0600] "GET /case-study/surfs-up-award-winning-new-sit

```

Contd.

Format	Use Case
CSV	Tabular data exchange
JSON	APIs, configuration, semi-structured data
Excel	Business and reporting data
Text	Logs, documents, raw data

2.2 Database access with relational database and non-relational database

You will learn:

- What databases are and why they are used
- Difference between:
 - Relational (SQL)
 - Non-relational (NoSQL)
- How Python interacts with databases

What is a Database?

- A database is an organized collection of data
- Designed for:
 - Persistent storage
 - Efficient retrieval
 - Secure access
- Databases allow multiple users/applications to access data simultaneously



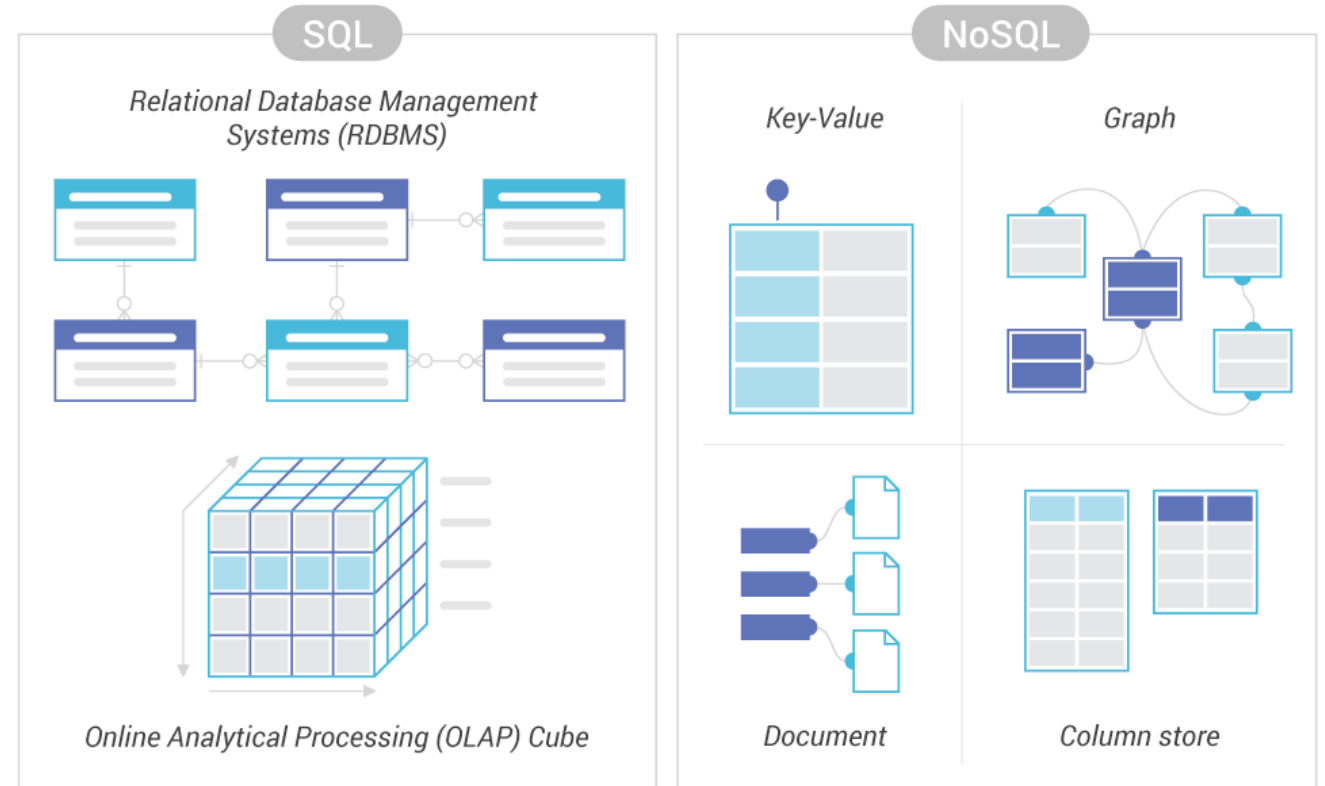
Why Use Databases Instead of Files?

Files	Databases
No concurrency control	Supports multiple users
Manual data management	Automated querying
No integrity constraints	Data consistency enforced
Not scalable	Designed to scale

Broad Classification of Databases

Databases can be broadly classified into:

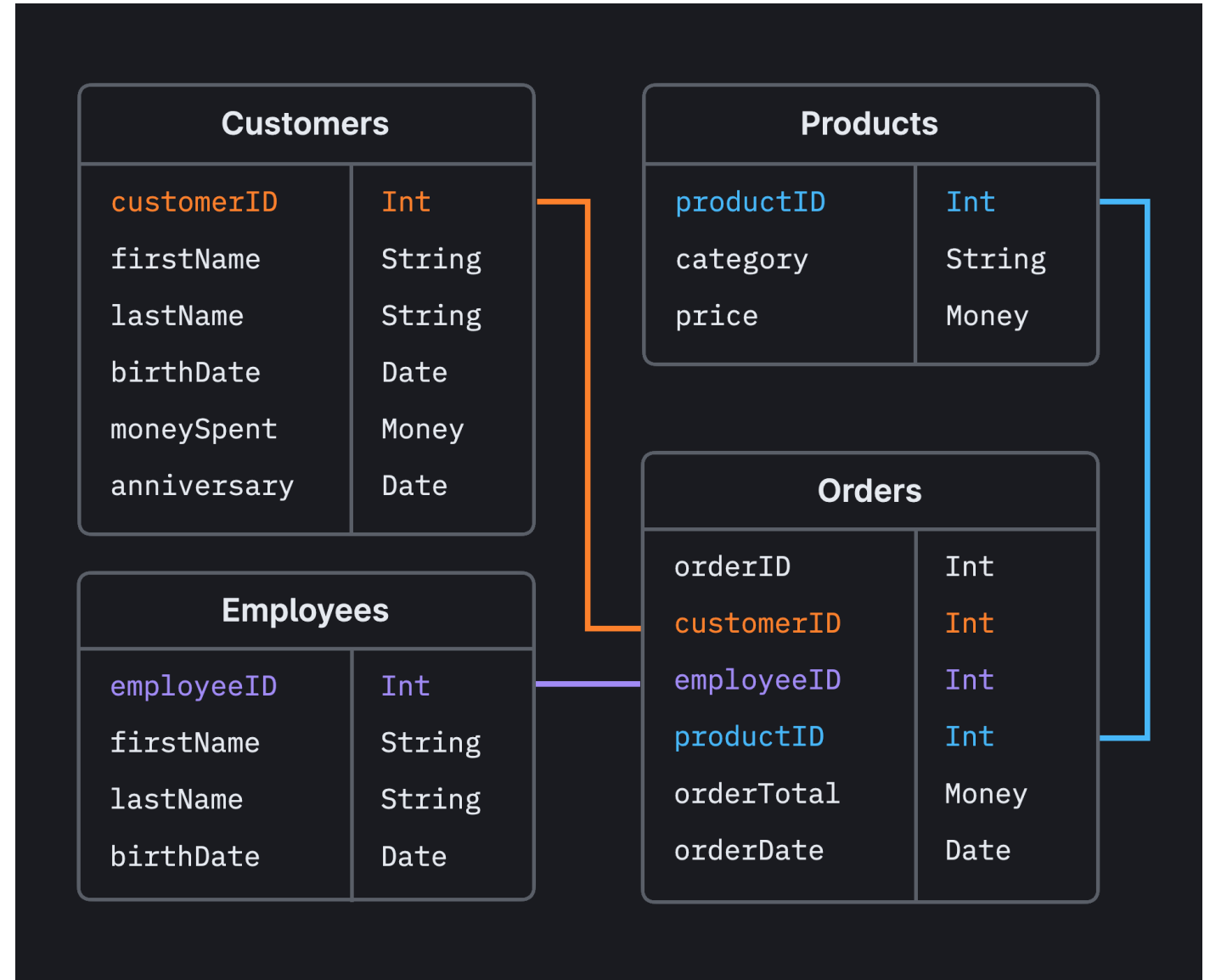
1. Relational Databases
2. Non-Relational (NoSQL) Databases



Relational Databases

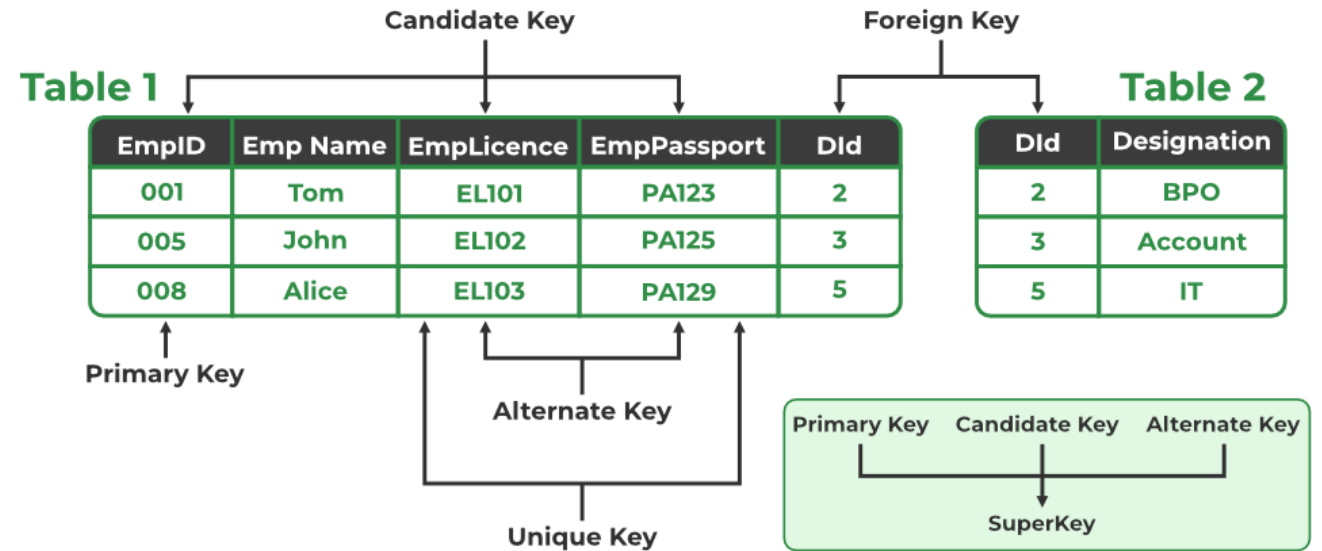
What is a Relational Database?

- Data stored in tables (relations)
- Tables consist of:
 - Rows (records)
 - Columns (attributes)
- Uses predefined schema
- Relationships established using keys



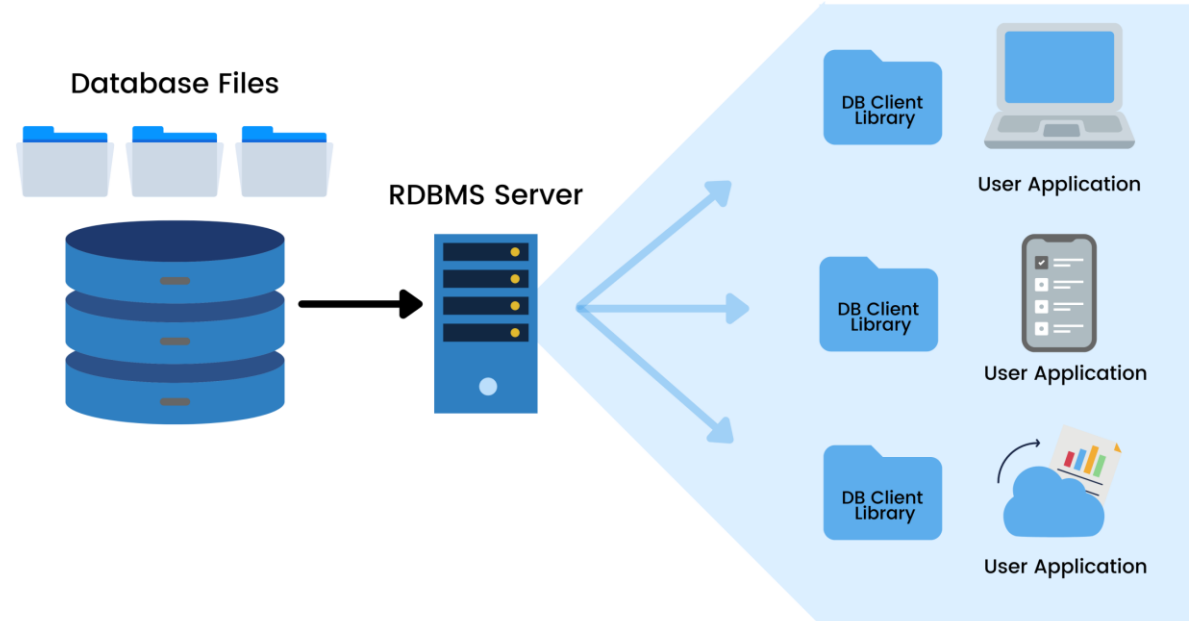
Key Characteristics of Relational Databases

- Fixed schema
- Structured data only
- Uses **SQL (Structured Query Language)**
- Enforces:
 - Data integrity
 - Constraints
 - Relationships



Examples of Relational Databases

- MySQL
- PostgreSQL
- Oracle
- SQLite



Non-Relational (NoSQL) Databases

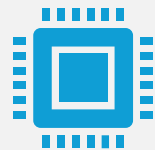


Stores data in non-tabular formats



Schema is:

Flexible
Dynamic



Designed for:

Scalability
High availability

Types of NoSQL Databases

Database	Type of NoSQL Database	Explanation
MongoDB	Document Database	Stores data as JSON-like documents with flexible schema
Redis	Key–Value Database	Stores data as key–value pairs, optimized for speed
Cassandra	Column-Family Database	Stores data in column families, designed for large-scale distributed systems
CouchDB	Document Database	Stores data as JSON documents, supports RESTful access

Key Characteristics of NoSQL Databases

Flexible schema

Horizontally scalable

Handles large volumes of data

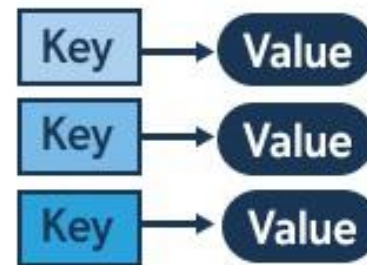
Optimized for performance

Examples of NoSQL Databases

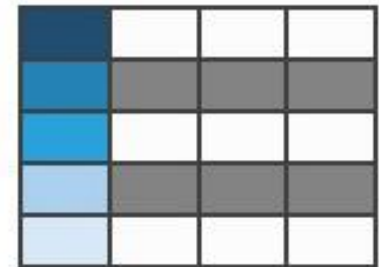
- MongoDB
- Redis
- Cassandra
- CouchDB

NoSQL

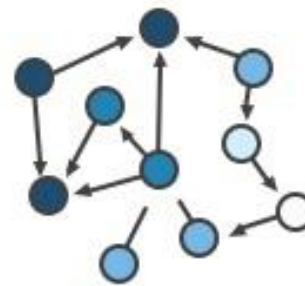
Key-Value



Column-Family



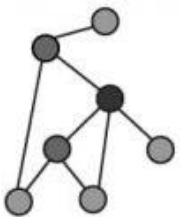
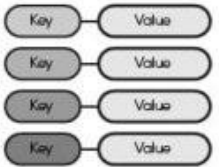
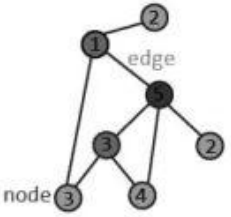
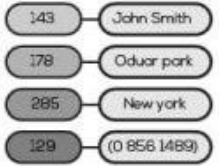
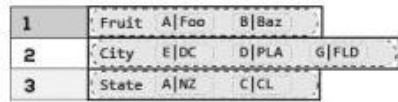
Graph



Document



NoSQL DATABASE TYPES

Document	Graph	Key-Value	Wide-Column
			
<pre>{ "user": { "id": "143", "name": "improgrammer", "city": "New York" } }</pre>			
			

Differences Between Relational and Non-Relational Databases

Aspect	Relational Database	Non-Relational (NoSQL) Database
Data Model	Table-based (rows and columns)	Non-tabular (document, key-value, column, graph)
Schema	Fixed, predefined schema	Flexible or schema-less
Data Type	Structured data only	Semi-structured or unstructured data
Query Language	Uses SQL	Uses API-based queries or query languages
Relationships	Explicit relationships using keys	Relationships handled at application level
Scalability	Vertical scaling (scale-up)	Horizontal scaling (scale-out)
Consistency	Strong consistency (ACID)	Eventual consistency (BASE)
Data Integrity	Enforced through constraints	Limited or no built-in constraints
Performance	Slower for very large, distributed data	Optimized for high performance and big data
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra

2.3 Accessing and processing data from APIs (REST, SOAP)

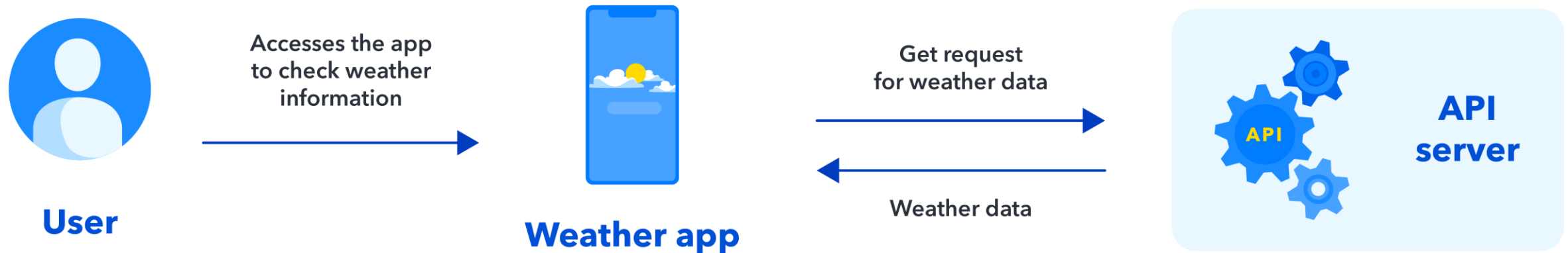
You will learn:

- What an API is
- How applications exchange data
- Difference between:
 - REST APIs
 - SOAP APIs
- How API data is processed in Python

What is an API?

- An API (Application Programming Interface) is a set of rules and protocols that allows two different software programs to communicate and exchange data with each other.
- It acts as a messenger or intermediary, taking a request from one application (the client) to another application or system (the server), and then delivering the response back.

API request and response flow diagram

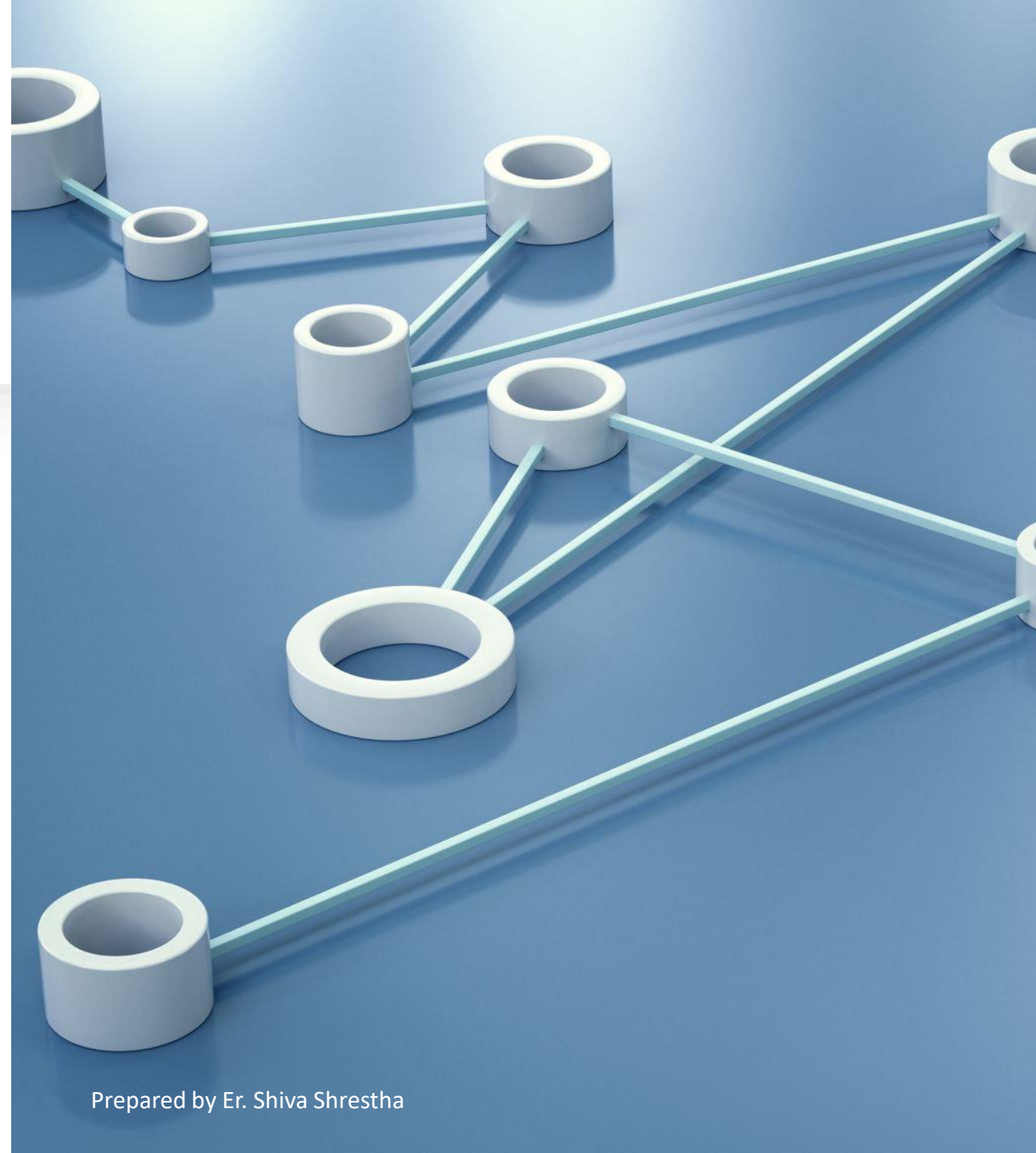


Why APIs Are Important in Data Science?

- Access real-time data
- Avoid manual data collection
- Integrate external services
- Scalable and automated

Examples:

- Weather data
- Financial data
- Social media data



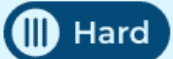
Types of APIs

APIs are commonly classified into:

- REST APIs
- SOAP APIs

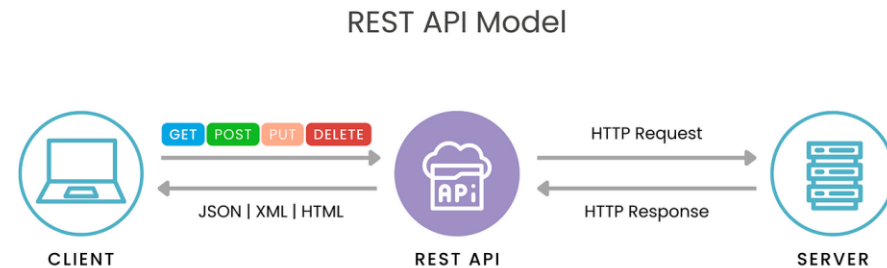


API ARCHITECTURAL STYLES

	RPC	SOAP	REST	GraphQL
Organized in terms of	local procedure calling	enveloped message structure	compliance with six architectural constraints	schema & type system
Format	JSON, XML, Protobuf, Thrift, FlatBuffers	XML only	XML, JSON, HTML, plain text,	JSON
Learning curve	 Easy	 Hard	 Easy	 Medium
Community	Large	Small	Large	Growing
Use cases	Command and action-oriented APIs; internal high performance communication in massive micro-services	Payment gateways, identity management CRM solutions financial and telecommunication services, legacy system support	Public APIs simple resource- driven apps	Mobile APIs, complex systems, micro-services

REST APIs

- A REST API (Representational State Transfer Application Programming Interface) is an architectural style for an API that uses **standard HTTP** requests to enable communication and data exchange between different computer systems over the internet.
- It is the most common approach for building web services due to its flexibility, scalability, and efficiency.





Characteristics of REST APIs

- Uses HTTP protocol
- Stateless
- Uses standard methods:
 - GET
 - POST
 - PUT
 - DELETE
- Data format: usually JSON

SOAP APIs

- The SOAP API (Simple Object Access Protocol Application Programming Interface) is a standards-based messaging protocol for exchanging structured information between applications over a computer network.
- It uses **XML** exclusively for its message format and relies on a formal contract defined by a WSDL (Web Services Description Language) file.

```
curl --location --request POST 'https://www.w3schools.com/xml/tempconvert.asmx' \
--header 'Content-Type: application/soap+xml; charset=utf-8' \
--data-raw '<?xml version="1.0" encoding="utf-8"?>
<soap12:Envelope
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:soap12="http://www.w3.org/2003/05/soap-envelope">
  <soap12:Body>
    <CelsiusToFahrenheit xmlns="https://www.w3schools.com/xml/">
      <Celsius>20</Celsius>
    </CelsiusToFahrenheit>
  </soap12:Body>
</soap12:Envelope>'
```



Characteristics of SOAP APIs

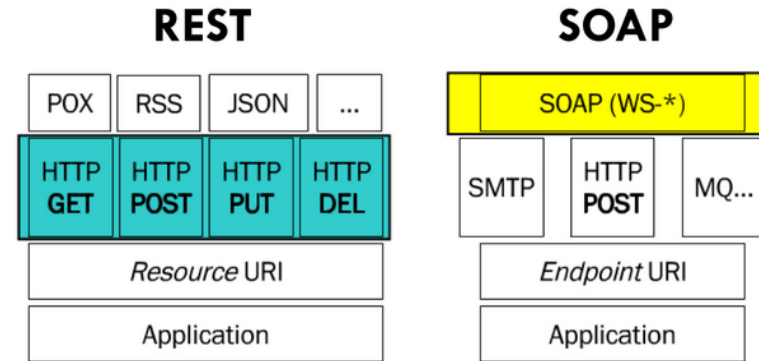
- XML-based
- Strict standards
- Verbose
- Common in legacy enterprise systems

Differences Between REST and SOAP

Aspect	REST	SOAP
Definition	Architectural style for web services	Protocol for web services
Data Format	JSON, XML, HTML (commonly JSON)	XML only
Complexity	Simple and lightweight	Complex and heavyweight
Communication	Uses HTTP directly	Uses HTTP, SMTP, TCP
Performance	Faster due to lightweight data	Slower due to XML overhead
Flexibility	Highly flexible	Less flexible
State	Stateless	Can be stateful or stateless
Security	Relies on HTTPS, OAuth	Built-in WS-Security
Error Handling	Uses HTTP status codes	Uses SOAP fault messages
Usage	Modern web and mobile applications	Enterprise and legacy systems

Summary

- **REST:** An architectural style that uses HTTP methods to access and manipulate resources.
- **SOAP:** A protocol that uses XML messaging to exchange structured information in web services.
- Tip To Remember:
 - REST → JSON → Lightweight → Modern
 - SOAP → XML → Heavy → Enterprise



2.4 Web scraping using requests and BeautifulSoup

You will learn:

- How to extract data from web pages
- When scraping is needed instead of APIs
- Tools used:
 - requests
 - BeautifulSoup
- Ethical considerations in scraping

What is web scraping?



What is Web Scraping?

- Technique to extract data from web pages
- Used when APIs are unavailable
- Converts HTML into structured data



Web Scraping (Exam-Oriented Definition)

- **Web scraping** is the automated process of extracting data from websites and converting unstructured or semi-structured HTML content into structured formats such as spreadsheets or databases for analysis and reuse.
- It typically uses automated programs (bots or crawlers) to collect large volumes of data that would otherwise require manual copy-and-paste.



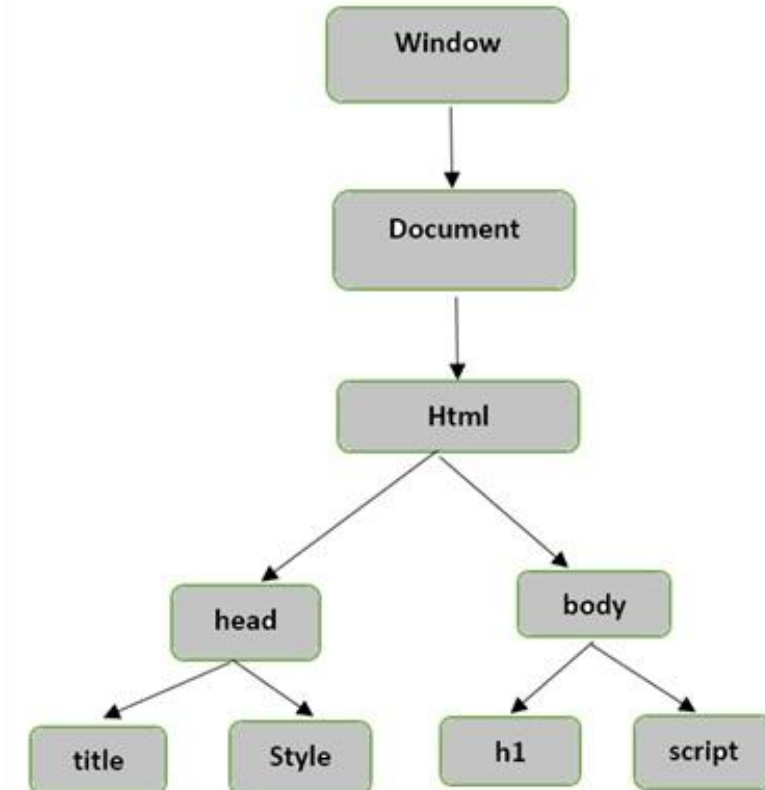
When to Use Web Scraping

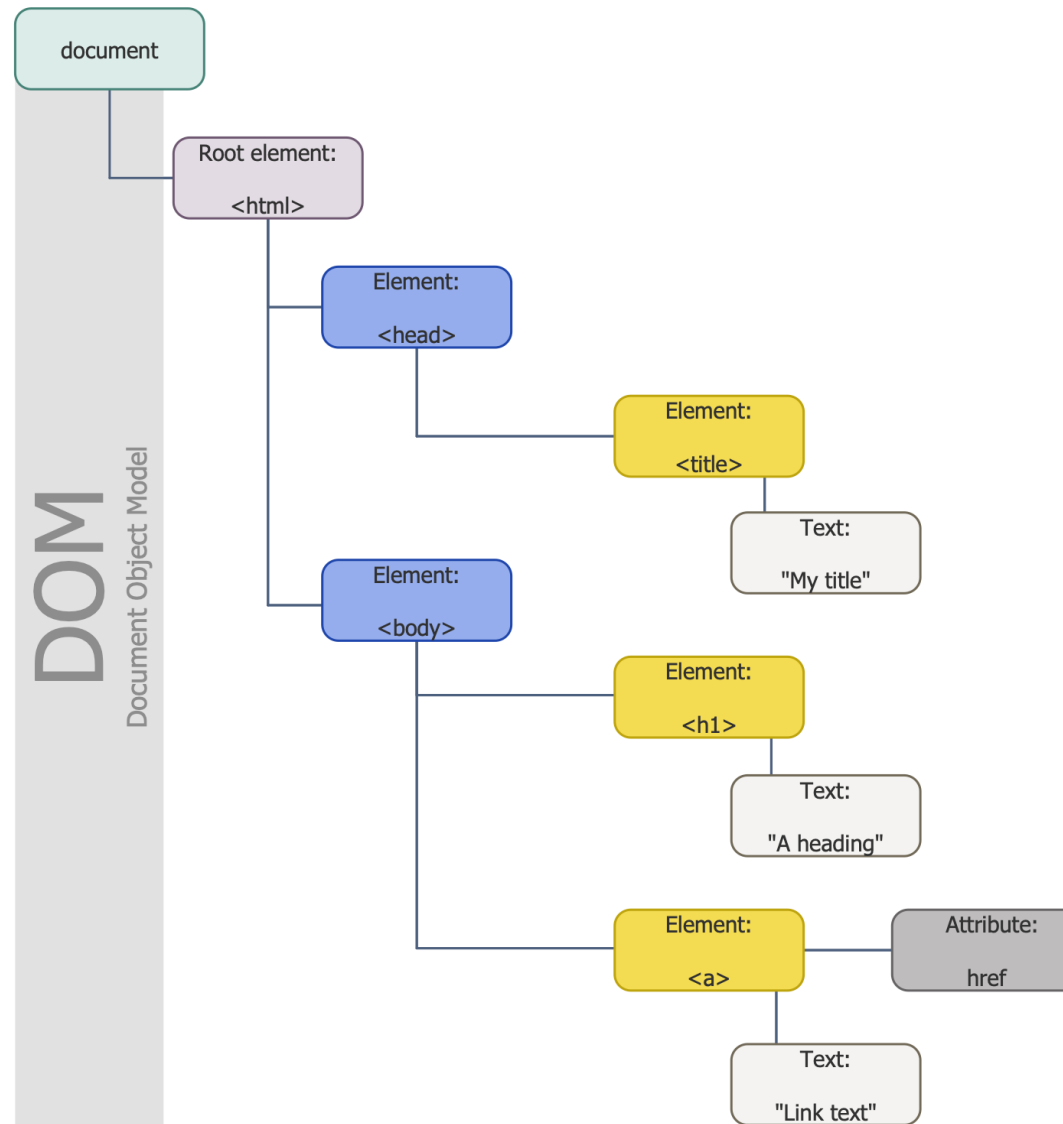
- Use web scraping when:
 - No public API exists
 - Data is publicly visible on websites
 - Manual copy is inefficient

Web Scrapping Tools in Python

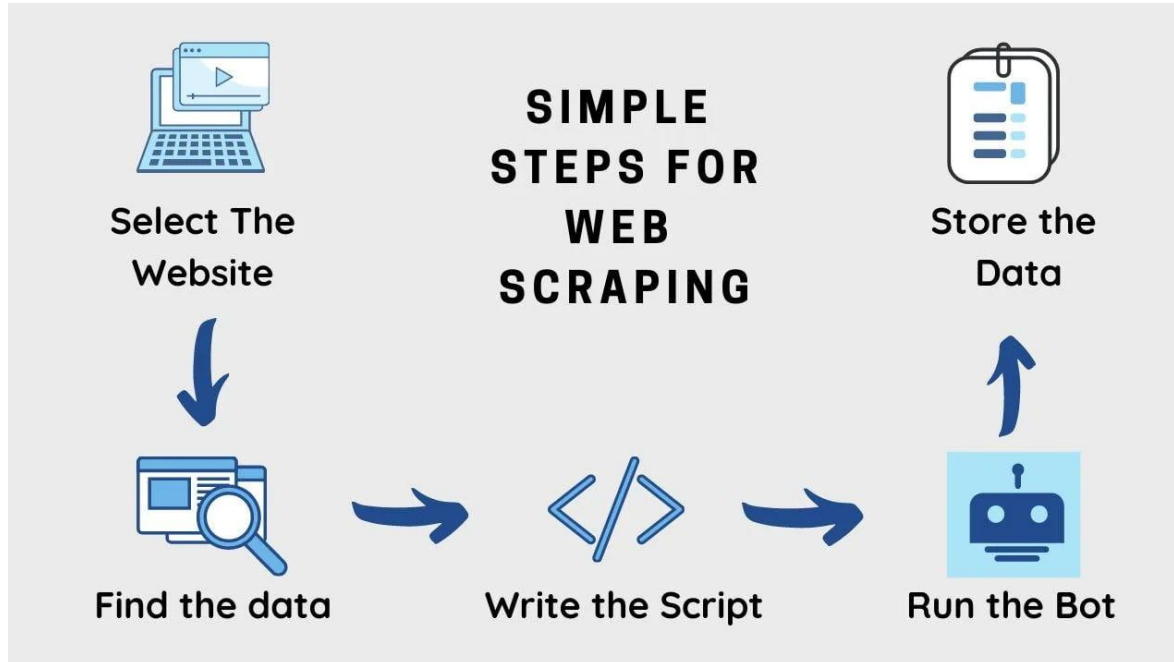
- **requests → fetch web page**
 - Request module allow us to easily handle HTTP in Python by sending HTTP requests and receiving responses with simple code.
- **BeautifulSoup → parse HTML**
 - Beautiful Soup is a library that makes it easy to scrape information from web pages. It sits atop an HTML or XML parser, providing Pythonic idioms for iterating, searching, and modifying the parse tree.

HTML Tree Structure





Web Scraping Workflow



1. Send HTTP request
2. Receive HTML response
3. Parse HTML DOM
4. Extract required elements
5. Store data



Uses of Web Scraping

Web scraping is commonly used for:

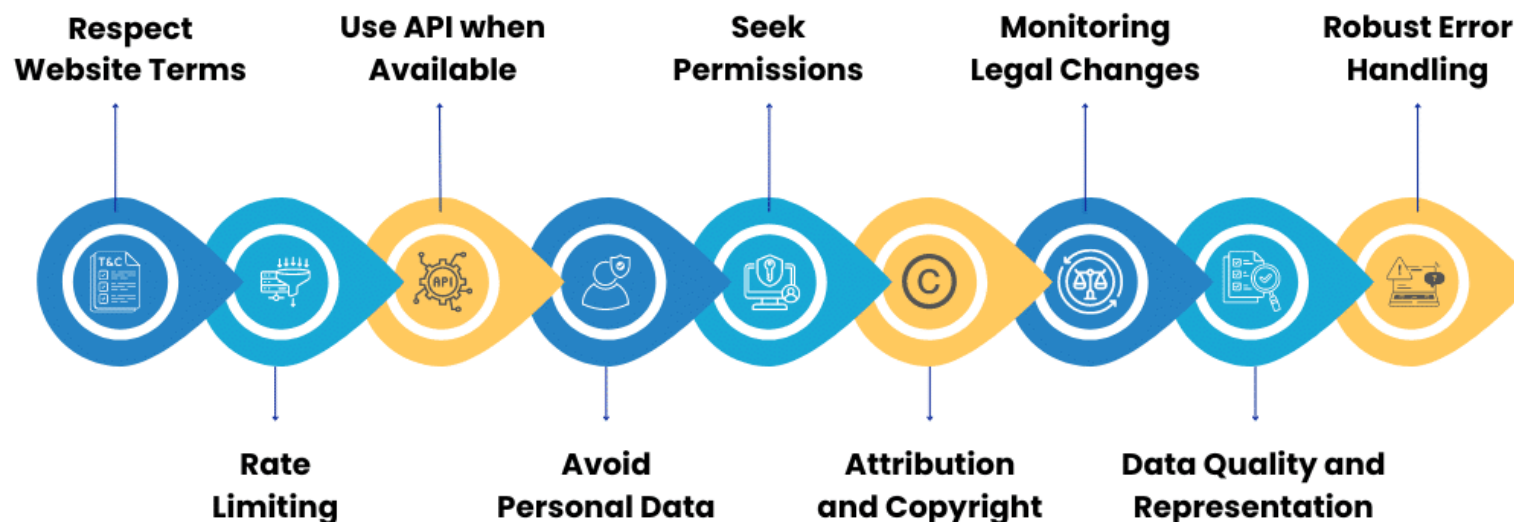
- Price monitoring
- Market research
- Lead generation
- Competitive analysis
- Training data for AI and machine learning models



Ethical and Legal Considerations

- Must comply with website **terms of service**
- Should respect **robots.txt**
- Should avoid excessive requests
- Must follow applicable legal guidelines

ETHICAL CONSIDERATIONS AND BEST PRACTICES FOR WEB SCRAPING



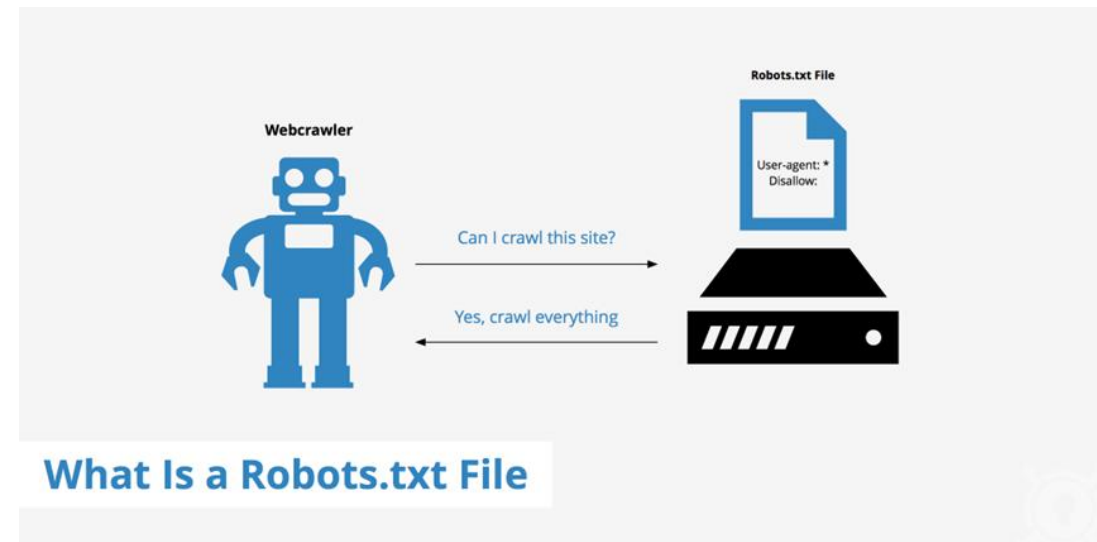


What is robots.txt?

- robots.txt is a **text file** placed at the **root of a website**
- It tells **web crawlers (bots)** which pages:
 - can be crawled
 - should not be crawled
- Used by **search engines and ethical scrapers**
- It is a **guideline**, not a security mechanism

Remember before Scraping

- **Allow everything:** All bots can access all pages
User-agent: *
Disallow:
- **Block everything:** No bot can access the site
User-agent: *
Disallow: /





Rule Explanation

- **User-agent: ***
 - The asterisk (*) means all web crawlers and bots
 - Applies to every user-agent (Googlebot, Bingbot, custom scrapers, etc.)
- **Disallow: (empty)**
 - An empty Disallow directive means nothing is blocked
 - Crawlers are allowed to access all pages

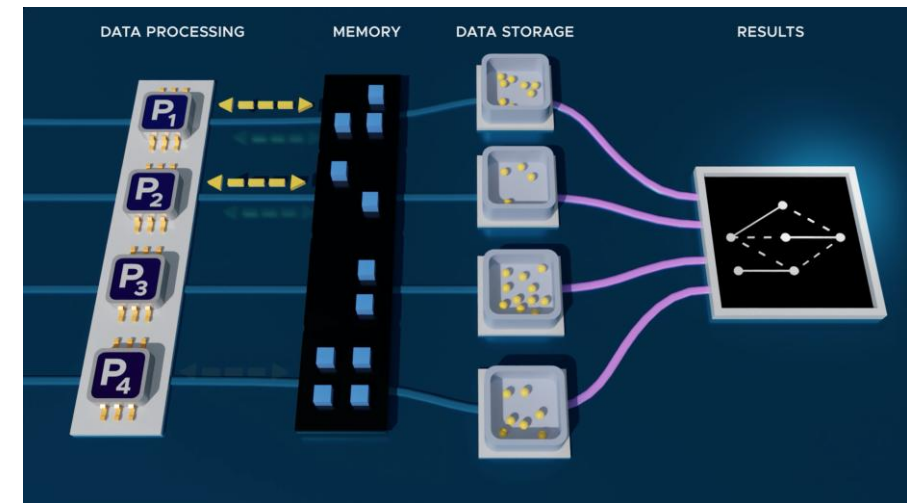
2.5 Handling large datasets with chunking and lazy evaluation

You will learn:

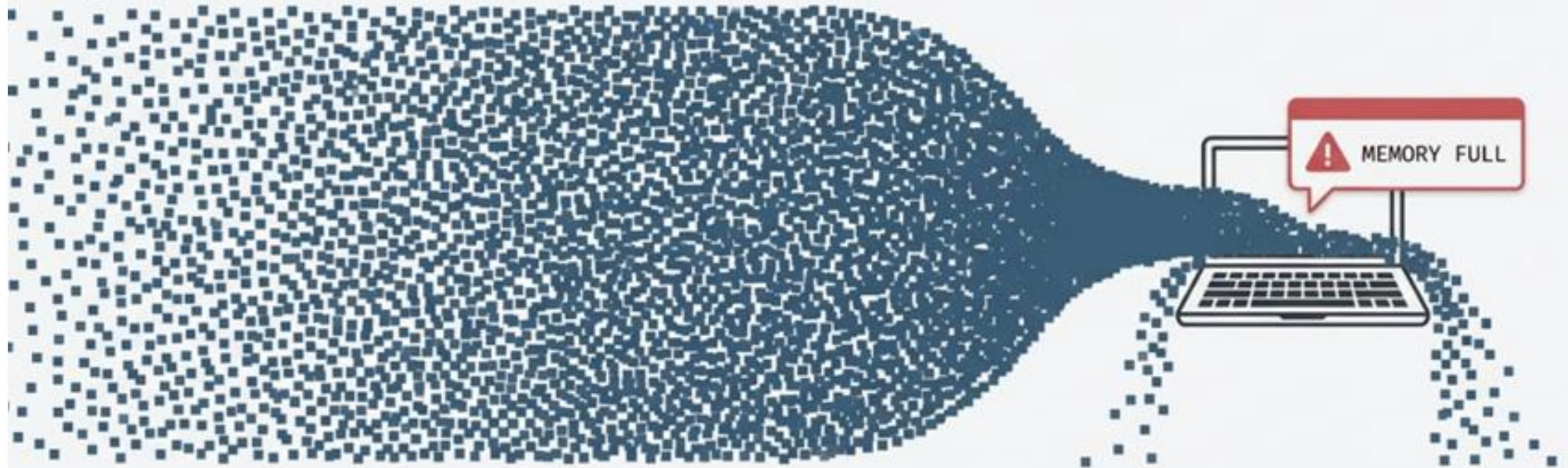
- Why large datasets are challenging
- How to process data:
 - In chunks
 - Lazily (on demand)
- How to avoid memory issues

Why Large Datasets Are Challenging

- Memory limitations
- Slow processing
- System crashes



The Scalability Challenge: When the Data is Too Big for Memory



The Problem

Whether from a high-volume API or scraping a massive site, you will encounter datasets that are gigabytes (or terabytes) in size. Attempting to load this data into a Pandas DataFrame all at once (`pd.read_csv(file)`) will consume all available RAM and crash your program or machine.

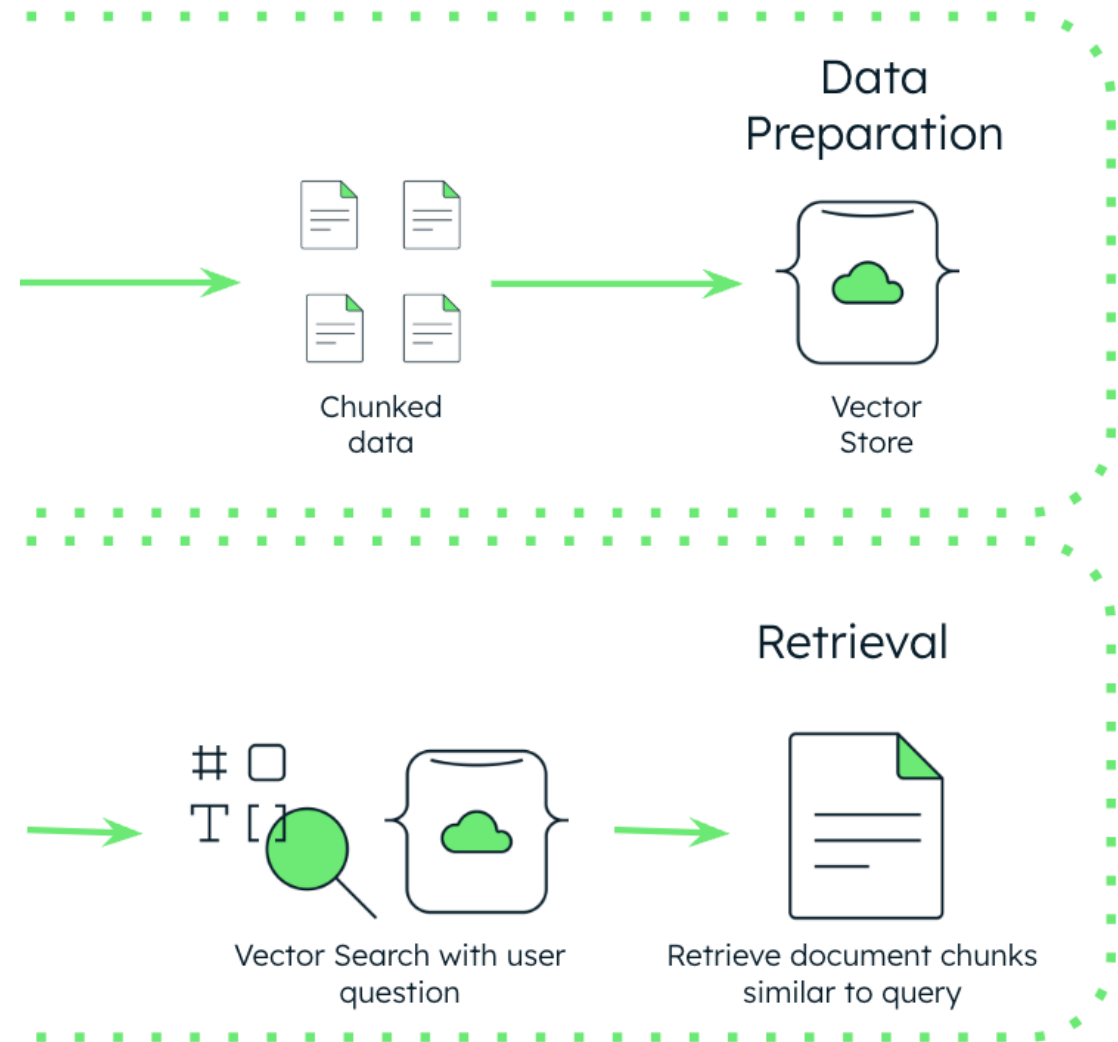
How do we process data that doesn't fit in system memory?

What is Chunking?

- Reading data in **small parts**
- Each part processed independently
- Reduces memory usage
- Chunking concept:

Large File

└─ Chunk 1 → process → discard
└─ Chunk 2 → process → discard
└─ Chunk 3 → process → discard

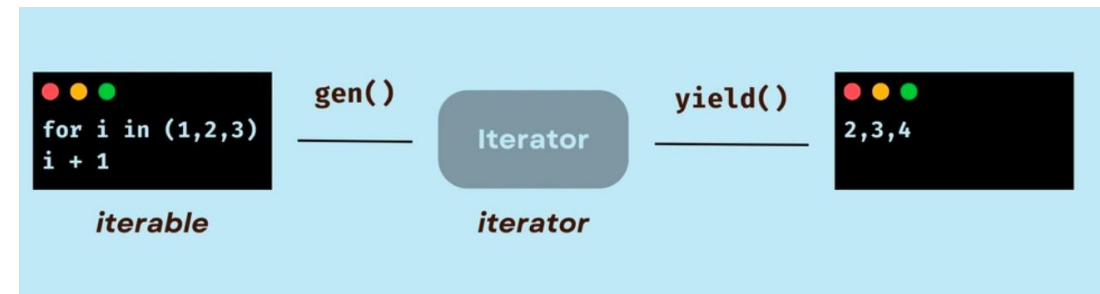


What is Lazy Evaluation?

- Computation delayed until needed
- Improves performance
- Saves memory

Examples:

- Iterators
- Generators



Chunking vs Lazy Evaluation

Aspect	Chunking	Lazy Evaluation
Definition	Processing data in fixed-size parts	Delaying computation until needed
Primary Goal	Reduce memory usage	Improve efficiency and performance
Data Handling	Data is divided into chunks	Data is generated or computed on demand
Memory Usage	One chunk in memory at a time	Minimal memory usage
Processing Style	Eager processing per chunk	Computation happens only when accessed
Typical Implementation	<code>chunksize</code> in file reading	Iterators and generators
Data Availability	Chunk exists as a full block	Values produced one at a time
Control	Programmer controls chunk size	Evaluation controlled by iteration
Common Use Cases	Large files (CSV, logs)	Streams, infinite sequences
Python Examples	<code>pandas.read_csv(chunksize=...)</code>	<code>yield</code> , generators, iterators



Real-World Applications

- Large CSV files
- Log processing
- Streaming data
- Data pipelines

In Summary:

- **Chunking:** A technique where large data is processed in smaller fixed-size blocks to save memory.
- **Lazy Evaluation:** A strategy where computation is postponed until the value is actually required.



Data Sources and APIs (Summary)

- Read and write data from **CSV, JSON, Excel, and text files**
- Access data from **relational and NoSQL databases**
- Fetch live data using **REST and SOAP APIs**
- Extract data from websites using **web scraping**
- Handle large datasets using **chunking and lazy evaluation**